

金融监管领域的 Harness 实践： 让知识与数据驱动 Agent 稳定运行

于浩军

总工程师

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon 上海站
全球软件开发大会

大模型正在重新定义软件
Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon 北京站
全球人工智能开发与应用大会

探索 AI 应用边界
Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

今天分享什么

AGENDA

01

应用场景

Agent 在金融监管中要解决什么

- 监管场景的四个硬约束
- 通用 Agent 的三大痛点
- 报送全链路场景全景

02

解决方案

Harness 体系如何支撑场景落地

- AIR Harness 智能体底座
- 知识资产 × 数据资产 × Agent
- 一次端到端实机演示

03

经验与展望

从试点到规模化的取舍

- 双向驱动飞轮
- 五个真实踩坑
- 下一步的三件事

01

应用场景

Agent 在金融监管中要解决什么

■ 金融监管报送：一个「不容 Agent 犯错」的场景

背景

特点 01

口径复杂

同一个「贷款余额」，在不同报表里口径不同：

统计范围、时点/时段、币种折算、抵消规则层层叠加

口径还会随监管制度修订而变

特点 02

频度高

日报、旬报、月报、季报并行

一次报送涉及数十张报表、上千个指标项

留给人工核对的窗口常常只有几个小时

特点 03

零容错

报错数据不是「体验问题」，而是合规问题

可能触发监管问询、通报、评级扣分

错误一旦报出，纠正成本远高于生成成本

特点 04

强追溯

每个数字都要能回答：谁算的、按哪条口径、取哪张表

监管核查时需要逐层还原计算过程

「结果对」不够，「过程可查」才算通过

通用 Agent 直接上场，会踩到的三个坑

痛点

痛点 01

口径漂移

「帮我算本月末小微企业贷款余额」

第一次按客户标签口径 → 128.6 亿；第二次按行业+规模口径 → 121.3 亿

根因：口径是从上下文「理解」出来的，不是从治理成果「取用」的

后果：报表内部勾稽对不上，越改越乱

痛点 02

幻觉

「这个指标的监管依据是什么」

回答引用了一份编号很规范的文件，实际该文号并不存在

根因：缺少受控的数据与知识入口，模型倾向「补全」而非「承认不知道」

后果：最危险的不是报错，而是「看起来很合理」的错

痛点 03

不可解释

「这个数为什么和上月差这么多」

给出了一段合理的归因，却说不出取了哪张表、过滤了哪些机构

根因：过程散落在对话里，没有结构化留痕

后果：无法通过监管核查，业务也不敢签字确认

共同根因：Agent 在「自由发挥」，而监管要求它「照章办事」——问题不在模型能力，在工程约束。

端到端场景全景：Agent 该介入的四个环节

场景全景



贯穿四个环节的同一条主线：每一步的口径来自治理成果、每一步的动作留下痕迹。

场景反过来给 Agent 提了四条硬性要求

第一部分小结

来自「口径复杂」

口径受控

不允许「发散口径」

只允许「取用已治理的口径」

来自「强追溯」

数据可查

每次取数走受控通道

取了什么、什么时候取，全部留痕

来自「零容错」

逻辑可溯

调用了哪个能力、引用了哪条规则

结果能逐层回溯到规则与数据源

来自「频度高」

能力可复用

高频监管动作不能每次重新推理

固化为标准能力，稳定复现

所以我们要做的不是「更聪明的 Agent」，而是「更受约束的 Agent」

第二部分讲的 Harness 体系，就是把这四条要求变成工程机制——而不是变成一段提示词里的「请务必注意」。

02

解决方案

Harness 体系如何支撑场景落地

AIR Harness 总体架构：四个场景、一个底座、两类资产

总体架构

定位：让 Agent 在监管「零容错、强追溯」要求下稳定运行的工程底座。



■ 执行模型：把自由的 Agent 循环改造成受约束的状态机

运行时 · 执行模型

通用 Agent 的循环是「想到什么做什么」；监管场景要求「只能在允许的动作集合里，按固定顺序做」。

通用 ReAct 循环

Thought: 自由推理下一步

Action: 任意工具 / 任意参数

Observation: 把返回塞回上下文

循环直到模型认为完成

问题：动作空间无界、步数无界、失败无界

受约束执行状态机

① Plan 只能从已注册 Skill 中选，产出结构化执行计划

② Guard 动作白名单 + 参数 schema 校验，越界直接拒绝

③ Execute Skill / Tool 执行，数值计算下推到引擎

④ Verify 输出契约校验 + 勾稽校验，不通过不进下一步

⑤ Trace 四要素落库，不留痕视为执行失败

任一环节不通过 → fail-closed: 停在此处转人工，不猜、不跳过

动作空间封闭

未注册的 Skill / Tool 不可调用，模型无法自造动作

步数与预算上限

单任务最大步数、最大 token、最大耗时，超限即中断

审批中断点

口径确认、数据补全、最终签发三处强制人工

失败即停

不做「再试一次换个说法」，异常直接暴露给人

■ 报送场景，Agent 在 Harness 里是怎么跑的

执行时序

这是业务视角的六个阶段；每个阶段内部都要跑一遍上一页的五状态循环（Plan → Guard → Execute → Verify → Trace）。



留痕轨道

每一步同步写入执行痕迹：Skill 版本 · 口径条款 · 数据来源与过滤条件 · 血缘路径 · 结果与耗时

整条链路上，Agent 只做「编排与判断」，不做「取数与定义」——后两件事必须走受控构件。

一个 Skill 的完整规格：口径是声明出来的，不是提示出来的

工程规格 · Skill

skill.yaml

```
skill: metric.calc.small_micro_loan_balance
version: 2.4.1                # 语义化版本，随口径变更递增
binding:                       # 与治理资产硬绑定，缺一即注册失败
  metric_id: M001
  caliber_ref: KI-M001-V7      # 指向知识条目 ID，不指向文档
  caliber_source: KB-0421#3.2  # 条目自带的原文出处，仅供人核对
  graph_node: (:Metric {id:'M001'})
inputs:
  as_of_date: date             # 报送时点
  org_scope:  enum[全行, 分行, 子公司]
  currency:   enum[CNY, 折人民币]
outputs:
  value:      decimal(18,2)
  unit:       亿元
  lineage:    ref              # 指向本次执行的血缘路径
execution:
  engine: sql                  # 数值计算下推引擎，不经过 LLM
  template: sql/m001_v7.sql.j2
  determinism: strict         # 同输入必得同输出
  idempotency_key: [metric_id, as_of_date, org_scope, caliber_version]
validation:                   # 本步输出契约，不通过则本步失败
  - consistency: value == sum(上游明细汇总)
  - range: [0, 1e12]
  # 报表级勾稽 A1.3 == A1.1 + A1.2 由校验类 Skill 负责
tests: cases/m001/*.yaml      # 回归用例，发布前必须全绿
```

口径引用是必填字段

caliber_ref / version 缺失时 Skill 无法注册，从机制上杜绝「口径写在提示词里」

数值计算不经过模型

engine: sql —— LLM 只负责填参数与路由，最终数字由引擎算出

契约化输入输出

参数有类型与枚举约束，输出有值域与勾稽校验，不通过即本步失败

幂等键与回归用例

同一幂等键重复执行结果一致；发布前跑 cases 全绿，差异必须逐条解释

口径受控执行：只允许取用，不允许理解

监管机制 · 对应痛点：口径漂移

要解决的问题： 同一个指标问两次给出两个数——因为口径是「理解」出来的，不是「取用」出来的。

机制 01

口径钉在治理成果上

指标计算逻辑、取数范围、过滤条件全部来自自己治理的知识与数据资产

Agent 只能引用，不能改写、不能推测

口径变更走治理流程，Agent 自动跟随新版本

机制 02

高频动作固化为 Skill

指标计算、报表勾稽校验等高频场景封装为标准 Skill

同样输入必得同样输出，可回归测试

Skill 带版本号，可追溯到某次报送用的是哪个版本

机制 03

口径缺失时明确「不知道」

找不到已治理口径时，不允许自行推导
转为待确认事项，交回业务或治理流程
宁可少答，不可错答

■ 我们把哪些监管动作固化成了标准 Skill

能力资产

判断标准很简单：高频、口径明确、结果必须每次一致的动作，一律固化，不交给模型即兴发挥。

口径类

- 口径解析与结构化
- 口径版本比对
- 口径适用性判定

让「按哪条口径」变成可查询的确定结果

计算类

- 指标计算
- 报表项汇总与折算
- 期初期末推导

同样输入必得同样输出，可做回归测试

校验类

- 表内勾稽校验
- 跨表一致性校验
- 值域/非空/波动校验

检核逻辑与监管要求同源，随版本演进

追溯类

- 血缘路径查询
- 变更影响分析
- 取数过程回放

把「说不清」变成一次图查询

输出类

- 质量分析报告
- 复核报告与审核建议
- 变更影响清单

结论可归档、可提交、可备查

Skill 带版本号与回归用例：某次报送用的是哪个版本、改动影响了哪些报表，都能查得到。

LLM 在这条链路里到底负责什么：边界清单

边界设计

把模型放在它擅长的位置——理解与表达；把它挡在它不可靠的位置——记账与判定。

允许 LLM 做

意图识别与任务拆解	把一句话请求变成结构化任务与报表项清单
槽位抽取	从监管文件中抽出指标、口径条件、频度、范围
Skill 路由与参数填充	选哪个 Skill、填哪些参数，但不决定口径本身
异常初判分类	把异常归类为数据缺失 / 口径差异 / 真实波动
文本生成	复核报告、处置说明、变更影响描述

禁止 LLM 做

数值计算与汇总	一律下推到 SQL / 计算引擎，模型不做算术
口径判定	口径只能从知识资产取用，不允许推导或补全
表名 / 字段名生成	只能从数据字典中选，选不到则报「未找到」
跳过或弱化校验	校验规则由引擎执行，模型无权决定是否通过
生成引用	文号、条款、版本必须来自检索结果，不允许拼接

还有一条容易被忽略的规则：所有对外输出的数值，必须直接引用引擎返回的结果对象，不允许模型转写。模型转写一次，就多一次「复述漂移」的机会——这类错误最难发现，因为格式看起来完全正常。

■ 受控连接：数据与知识只走唯一授权入口

监管机制 · 对应痛点：幻觉

要解决的问题： 编造字段、编造文号、编造表——本质是缺少受控的数据与知识入口。



效果：Agent 想「编」也编不出来——它拿不到未经授权的数据，也无法绕过入口自行生成事实。

■ 全链路可追溯：不留痕的动作视为执行失败

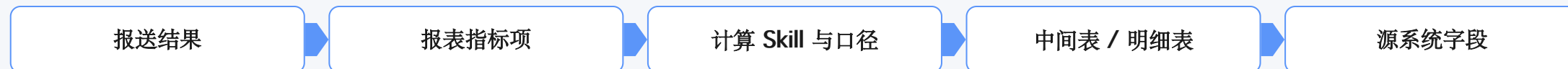
监管机制 · 对应痛点：不可解释

要解决的问题：数出来了但说不清怎么来的——过程散落在对话里，没有结构化留痕。

每一步执行都记录四要素，形成可逐层回溯的证据链：



任一数字都可以这样逐层回溯：



追溯不是「事后补日志」，而是执行链路的一等公民——不留痕的动作，直接视为执行失败。

一条执行留痕，实际长什么样

可追溯 · 具象化

以「某月报 A1.3 项 = 128.6 亿」为例，系统留下的是这样一条结构化记录（示意）：

执行 ID / 任务	RPT-2026M07-A1.3-0007 · 2026 年 7 月 月报报送
调用 Skill	指标计算 Skill v2.4.1（回归用例通过，最近变更 2026-06-18）
引用口径	知识条目 KI-M001-V7（出处：《XX 报表填报说明》第 3.2 条 · KB-0421）
数据来源	dw.f_loan_balance（分区 20260731）· 过滤：机构=全行、客户类型 IN（小微）
血缘路径	S_LOAN 明细 → T_LOAN_SUM 中间表 → M001 指标 → A1.3 报表项
勾稽关系	A1.3 = A1.1 + A1.2（校验通过，差额 0.00）
结果与耗时	128.6 亿元 · 计算耗时 2.3s · 检核 12 条全部通过
经办与复核	系统自动生成 → 张某某复核确认 → 李某某签发

监管核查时，我们交付的不是一句解释，而是这样一条可以逐字段核验的记录。

■ 知识资产与数据资产：Agent 唯一被允许的两个事实来源

资产视图

Agent 不生产事实。它要么从知识资产取「是什么、怎么算」，要么从数据资产取「实际是多少」。



两类资产互为因果：数据资产里的「定义」沉淀为知识资产；知识资产里的「口径」决定数据资产怎么加工。
注意边界：制度文件的原件属于数据资产（非结构化）；只有抽取并经专家确认的结构化条目，才算知识资产。

关系逻辑：Agent 的每一步动作，分别吃哪一类资产

供给关系

把 Agent 的动作逐个拆开看，就能发现：每一步都必须有对应的资产托底，否则必然退化成幻觉或漂移。

Agent 的动作	依赖的知识资产	依赖的数据资产	资产缺位时会退化成什么
理解监管要求	监管制度原文、报送指引、条款版本	历史报送结果与现行报表模板（比对差异）	凭印象解读条款，张冠李戴
确定口径并计算	指标口径与计算逻辑（含版本、折算抵消规则）	明细类、指标类数据	自行推导口径 → 口径漂移、结果不可复现
生成取数逻辑	表 / 字段定义、数据字典、数据标准	明细类数据及其元数据	编造字段名与表名 → 幻觉
校验勾稽关系	勾稽关系图、跨报表依赖	报表类数据、上期报送结果	只能做单点检查，发现不了跨表矛盾
解释与追溯	血缘关系图、口径版本、执行留痕	各层数据及其加工记录	只能给出「看起来合理」的归因

结论：第一部分的三大痛点，本质上都是「某一类资产没有到位」的表现。
所以治理不是 Agent 的前置项目，而是 Agent 的运行时依赖——它每一秒都在消费这些资产。

图里到底存了什么：一个指标的勾稽与血缘子图

图数据库 · 具象化

节点是报表项 / 指标 / 表 / 字段，边是「计算」「汇总」「勾稽」「依赖」——都是可查询的结构化事实。

血缘关系（数据从哪来）



勾稽关系（和谁必须对得上）



这张图被用来做三件事

- 异常时反向定位断点
- 变更时正向评估影响范围
- 为 Agent 提供计算路径上下文

有了这张图，「这个数怎么来的」和「改这里会影响谁」都不再依赖个人经验。

■ 知识资产怎么生产：一条带人工关口的流水线

知识工程 · 流程

知识资产不是「把文件丢进向量库」，而是一条有抽取、有确认、有版本、有回流的流水线。



回流：Agent 在执行中遇到的口径歧义、缺失定义、条款冲突，自动登记为知识缺口，回到第 ① 步

硬规则：只有经过专家确认的知识才能被 Agent 引用
未确认的抽取结果只进「待确认池」，不参与任何执行

带来的效果：知识越用越多、越用越准
口径变更只需改一处，所有引用它的 Agent 行为同步跟随

向量检索在监管口径上的五个硬缺陷，分别用什么补

检索架构 · 问题定义

向量适合「找相关」，监管口径要的是「找唯一且当期生效的那一条」——这是两件不同的事。

向量的缺陷	在监管场景的具体表现	我们用什么机制补上
近义不同义	「小微企业贷款」按企业划型标准认定，「普惠型小微企业贷款」还需满足单户授信总额上限——两者向量距离极近，口径	术语本体 + 同义/易混词典：查询先做术语归一；易混对强制走精确键或反问确认
版本无感	不同一条口径 V6 与 V7 文本相似度 0.98，召回时几乎无法区分	版本与生效期前置过滤（as-of 查询）：不是找最像的，是找当期生效的
精确定位弱	「A1.3 项怎么填」本质是主键查找，向量给的是相关段落，不是那一条	知识条目化 + 主键直取：（报表项，期间）→ 条目 ID，命中即返回，不进向量
不表达关系与约束	能召回「是什么」，答不了「和谁勾稽、由谁算出、改了影响谁」	图数据库承载关系；向量候选必须通过图约束校验才允许采纳
相似度不等于置信度	0.83 分到底能不能用？分数没有绝对含义，且随语料更新而漂移	双阈值门限（绝对分数 + top1/top2 margin）+ 拒答；低置信直接进待确认池

一句话定位：向量负责「找到候选」，不负责「决定答案」——决定答案的是主键、版本、图约束和门限。

知识库里存的不是文档切块，而是结构化口径条目

知识建模 · 最小单元

knowledge_item.yaml

```
item_id: KI-M001-V7          # 条目主键，可被 Skill 直接引用
type: metric_caliber
status: active                # draft→pending→active→deprecated
subject:
  metric_id: M001
  metric_name: 小微企业贷款余额
  report_scope: [A1.3, B2.1] # 适用报表项
effective:
  from: 2026-01-01
  to: null                    # null = 当期生效
caliber:
  population: 按《中小企业划型标准》认定的小型、微型企业
  time_point: 报送时点期末余额
  filters: [排除表内贴现, 排除委托贷款]
  currency_rule: 外币按报送日中间价折人民币
confusable_with:              # 易混对显式声明，检索时强制消歧
  - KI-M014-V3                # 普惠型小微：单户授信总额 <= 1000 万
source:
  doc: KB-0421
  clause: '3.2'
  quote_anchor: p12#para3     # 原文锚点，供人核对
graph: (:Metric{id:'M001'})-[:DEFINED_BY]->(:CaliberItem{id:'KI-M001-V7'})
embedding: derived            # 向量是派生物，不是主体
```

主键可被直接引用

Skill 的 caliber_ref 指向条目 ID，而不是指向一份文档或一个 chunk

状态机管生命周期

draft → pending → active → deprecated，只有 active 允许被引用

易混对写进条目本身

confusable_with 把最容易搞错的邻居显式声明，检索命中即触发消歧

向量是派生物

条目变更后重建向量；向量丢了只影响可发现性，不影响正确性

检索路由：能精确取的，绝不用相似度

检索架构 · 五级降级

五级依次尝试，命中即返回；向量只出现在第 4 级，而且它的输出还要过第 5 级才算数。

L1 主键直取

(报表项 / 指标 + 期间) → 条目 ID，确定性查询，零歧义

L2 术语归一后重取

同义词 / 别名 / 缩写映射到标准术语，再回 L1

L3 精选问答集

人工维护的高频口径问答，命中即直出，不进模型

L4 混合召回

BM25 + 向量双路 → RRF 融合 → cross-encoder 重排

L5 图约束校验

候选必须挂在目标节点且当期生效，否则一律丢弃

— 都不命中：拒答

返回「未找到」，同时登记知识缺口回流生产流水线

采纳门限（同时满足才允许作答）

- 绝对分数 $\geq$ 阈值（按域分别标定）
- top1 与 top2 的 margin $\geq$ 阈值（防两条口径难分）
- 通过 L5 图约束校验

任一条不满足 → 不作答，转待确认池并记录 miss 原因：无条目 / 多条冲突 / 低置信

Cypher · 约束校验

```
// L5 采纳前的图约束校验
MATCH (m:Metric {id:$metric})
  -[:DEFINED_BY]->(c:CaliberItem)
WHERE c.id IN $candidates
  AND c.status = 'active'
  AND c.from <= $as_of
  AND (c.to IS NULL OR c.to > $as_of)
RETURN c.id, c.version
```

知识库自己也要被体检：五个健康度指标

知识治理 · 质量闭环

检索机制再好，也救不了一个有缺口、有过期、有冲突的知识库。所以知识库本身必须被持续度量。

健康度指标	怎么算	不达标怎么处置
覆盖率	由报表项 × 指标清单推导出「应有条目」，与已有 active 条目比对	缺口清单进排期；报送前缺口未补则该项转人工
鲜度	条目引用的文件版本与最新监管文件版本比对	过期条目告警；阻断级：相关报表不允许自动报送
冲突	同一（指标，期间）存在多条 active 条目，即存在二义	阻断级：必须由专家裁决保留哪一条，不允许并存
孤儿条目	没有任何指标 / 报表引用它，也没有被检索命中过	清理或补挂到正确节点，避免污染召回结果
引用命中率 / miss 分布	Agent 执行中命中与未命中的比例，及 miss 原因分布	miss 自动登记为知识缺口，回流到知识生产流水线

这套体检每期报送前跑一次：冲突与过期是阻断级，覆盖缺口进排期——知识库的质量不靠人记得住。

数据资产怎么治理：一边加工，一边把定义沉淀成知识

数据工程 · 流程

四层加工链上，每一层都同时做三件事：把数据加工出来、把治理动作做掉、把定义沉淀成知识。



沉淀为知识资产的三类定义：

- ① 表 / 字段定义 ② 指标口径与计算逻辑 ③ 报表口径、频度与范围
——数据治理的产物直接成为 Agent 的执行依据，这就是「数据即知识」。

知识、数据与 Agent 的关系闭环

总关系图



正向: 资产供给 Agent, 决定它每一步能不能做对; 反向: Agent 反哺资产, 让治理越做越轻。

怎么量化「稳」：上生产之前必须过的三道关

评测与度量

在监管场景里，「准确率 95%」不是一个可以汇报的指标——要么这一项能报，要么它必须转人工。

第一关 · 红线用例

- 口径歧义、跨表勾稽、极端值与空值
- 字段幻觉率必须为 0
- 任一红线不过，一律不允许发布

第二关 · 回归基线

- 每个 Skill 绑定历史真实报送数据
- 口径或实现变更后跑全量回归
- 差异逐条解释，不接受「大致相同」

第三关 · 灰度双轨

- 与人工流程并行若干报送期
- 只观察不接管，比对差异并复盘
- 稳定后再逐步收敛人工投入

度量指标	定义	目标与红线
口径命中率	已作答的执行中取用了已治理口径的占比	100%（未命中一律不作答，不进入这个分母）
口径版本正确率	引用版本与报送期应适用版本一致	100%（版本错即视为报错）
字段幻觉率	输出中出现不存在的表 / 字段 / 文号	0（红线指标，出现即阻断发布）
应拒答场景拒答率	资产缺位时正确回答「未找到」的比例	越高越好，低于阈值不允许上量
回归差异条数	版本变更后与基线不一致的条目数	允许非零，但每条必须有解释与结论

■ 实机演示：一次完整流程，从数据采集到报送完成

端到端演示

幕 1 需求侧 导入一份新版报送要求，自动产出要求清单、取数逻辑与检核规则

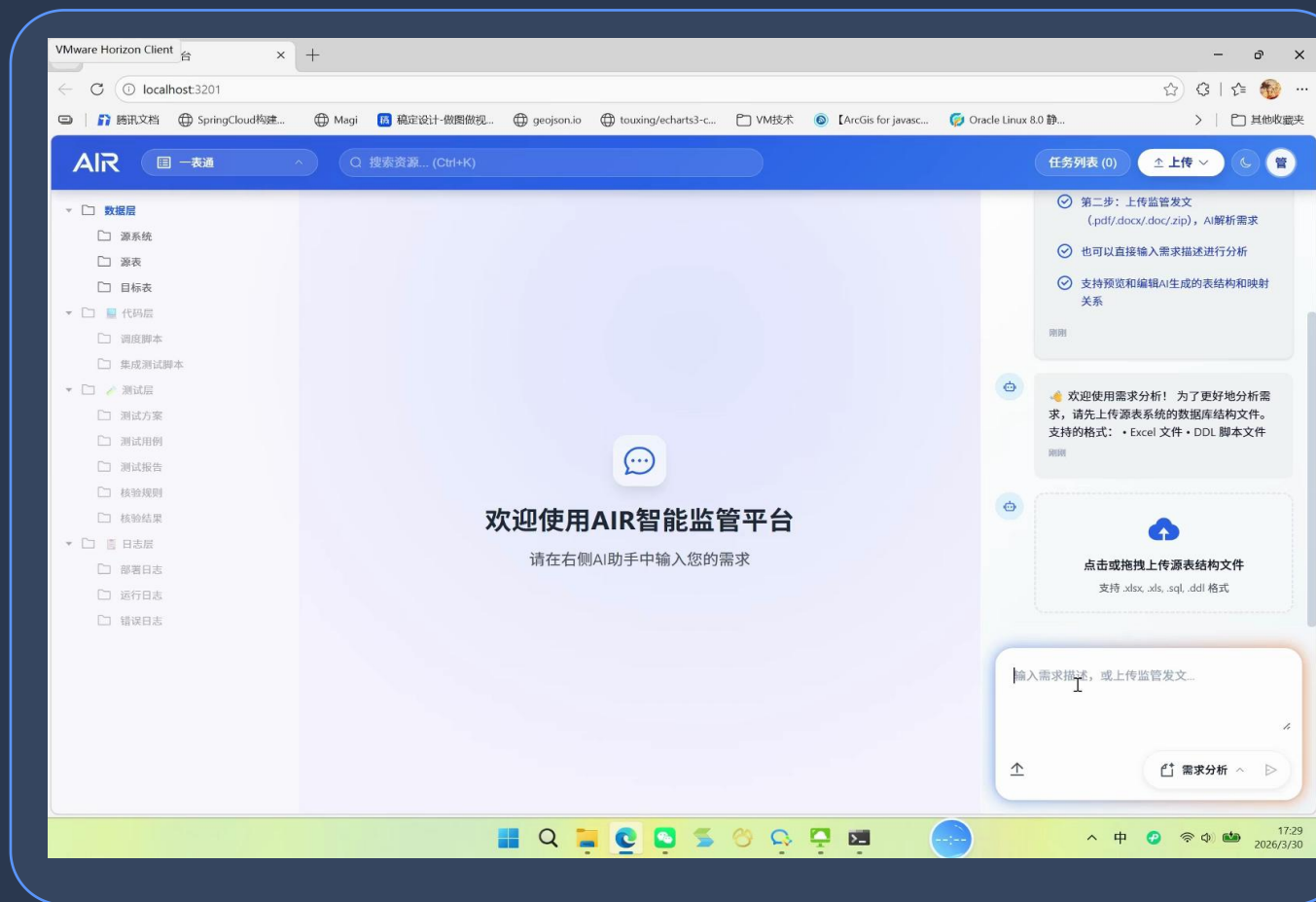
幕 2 报送中 一键智能初始化，触发四层自动检核（单元格 → 勾稽 → 跨表 → 趋势）

幕 3 异常拦截 勾稽差额被自动拦下，一键根因分析给出分类结论与证据

幕 4 血缘追溯 沿血缘图上溯定位到源系统字段，给出补全方案与依据

幕 5 复核出具 生成复核报告与审核建议，按风险等级排序待确认事项

兜底：若现场环境异常，切换到已录制的全流程录屏，按同样的六幕讲解。



03

经验与展望

从试点到规模化，我们学到了什么

双向驱动：治理与智能互为前提

落地方法论

不是「先把治理做完再上 Agent」，而是让两件事在同一个闭环里互相加速。



◀ 闭环回流：每一次报送同时沉淀两样东西——更准的治理成果，和更稳的 Agent 能力

不要等治理「做完」再上 Agent，也不要指望 Agent 替代治理——从一张最难对的报表起步，让它们互相拉着往前走。

下一步：把「稳」做成可复制的能力

展望

方向 01

Skill 资产化

把跨机构通用的监管动作沉淀为标准 Skill 库

带口径版本、带回归测试用例

新报表上线从「开发」变成「组装」

方向 02

从影响清单到自动改造

今天能自动算出影响范围，下一步要自动给出改造方案

变更后自动触发受影响 Skill 的计算差异

把「逐张报表人工核对」压缩成「只确认」

方向 03

可信度量与评估

为 Agent 的每次执行给出可信度与证据完整性评分

建立面向监管场景的评测集与红线用例

让「能不能上生产」有客观标准

监管场景不需要最聪明的 Agent，需要每次都一样、每步都说得清的 Agent。

而这件事，靠的是知识与数据的持续治理，加上一个把约束做实的 Harness。

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会